

Anexa C

Codul Sursa de Simulare si Interfatare (Python)

```
1 import sys
2 import os
3 import serial
4 import time
5 import struct
6 import binascii
7 import random
8
9 # ==== SUMO CONFIGURATION ==== #
10 SUMO_TOOLS = r"C:\Program Files (x86)\Eclipse\Sumo\tools"
11 if SUMO_TOOLS not in sys.path:
12     sys.path.append(SUMO_TOOLS)
13
14 import traci
15
16 SUMO_BINARY = "sumo-gui"
17 CONFIG_FILE = "mysim.sumocfg"
18 SIM_DURATION = 3600
19 TL_ID = "TL_WEST"
20 TL_EAST_ID = "TL_EAST"
21 DEBUG = False
22 FREQUENCY_CONTROL = False
23
24 # ==== SERIAL CONFIGURATION ====
25 SERIAL_PORT = 'COM22'
26 BAUD_RATE = 9600
27 CHUNK_SIZE = 8
28
29 START_STOP_DELIMITER = 0xFF
30 PADDING_BYTE = 0x00
31 RECEIVE_TIMEOUT = 5
32
33 # ==== LANE & MAPPING DEFINITIONS ====
34
35 # Maps Lane IDs(0-16) to SUMO Lane ID Strings
36 TLWest_Lane_Dictionary = {
37     0: "W_POP1P2_Downstream_1",
38     1: "W_POP1P2_Downstream_2",
39     2: "W_PO_UPSTREAM_3",
40     3: "W_PO_UPSTREAM_2",
41     4: "W_PO_UPSTREAM_1",
42     5: "E5_1",
43     6: "E5_2",
44     7: "S_P2_UPSTREAM_4",
45     8: "S_P2_UPSTREAM_3",
46     9: "S_P2_UPSTREAM_2",
47     10: "S_P2_UPSTREAM_1",
48     11: "W_POP2_Downstream_E_POP1_Upstream_1",
49     12: "W_POP2_Downstream_E_POP1_Upstream_2",
50     13: "W_POP2_Downstream_E_POP1_Upstream_3",
51     14: "W_POP1_Upstream_E_POP2_Downstream_3",
52     15: "W_POP1_Upstream_E_POP2_Downstream_2",
53     16: "W_POP1_Upstream_E_POP2_Downstream_1"
54 }
55
56 TLEast_Lane_Dictionary = {
57     0: "N_P1_Upstream_1",
58     1: "N_P1_Upstream_2",
59     2: "N_P2_Upstream_4",
60     3: "N_P2_Upstream_3",
61     4: "N_P2_Upstream_2",
62     5: "N_P2_Upstream_1",
63     6: "W_POP1_Upstream_E_POP2_Downstream_1",
64     7: "W_POP1_Upstream_E_POP2_Downstream_2",
65     8: "W_POP1_Upstream_E_POP2_Downstream_3",
```

```

66     9: "W_POP2_Downstream_E_POP1_Upstream_3",
67     10: "W_POP2_Downstream_E_POP1_Upstream_2",
68     11: "W_POP2_Downstream_E_POP1_Upstream_1",
69     12: "E_POP2_Downstream_1",
70     13: "E_POP2_Downstream_2",
71     14: "E_PO_Upstream_2",
72     15: "E_PO_Upstream_1"
73 }
74
75 TL_West_Downstream_Lanes = [0,1,5,6,11,12,13]
76 TL_East_Downstream_Lanes = [12,13,6,7,8,0,1]
77
78 # Mapping SUMO Signal Index -> Hardware Lane ID
79 TL_West_sumo_to_hw_index = [16, 15, 14, 7, 7, 9, 10, 10, 4, 3, 2, 2]
80 TL_East_sumo_to_hw_index = [2, 3, 4, 5, 5, 14, 14, 15, 15, 11, 10, 9]
81
82 # ==== HELPER FUNCTIONS ==== #
83
84 def build_transmission_packet(values):
85     """Encodes and chunks the traffic values for transmission"""
86     encoded_data = [val + 1 for val in values]
87     full_message_body = [START_STOP_DELIMITER] + encoded_data + [START_STOP_DELIMITER]
88
89     message_length = len(full_message_body)
90
91     padding_needed = (CHUNK_SIZE - (message_length % CHUNK_SIZE)) % CHUNK_SIZE
92     padded_message = full_message_body + [PADDING_BYTE] * padding_needed
93
94     bytes_to_send = bytes(padded_message)
95     if(DEBUG):
96         print(f" [TX] Original Size: {message_length} bytes. Padded to: {len(
97             bytes_to_send)} (Chunks of {CHUNK_SIZE})")
98     return bytes_to_send
99
100 def send_data_chunks(ser, bytes_to_send):
101     """Sends the prepared byte sequence in CHUNK_SIZE byte chunks."""
102     for i in range(0, len(bytes_to_send), CHUNK_SIZE):
103         chunk = bytes_to_send[i:i + CHUNK_SIZE]
104         ser.write(chunk)
105         # We need this sleep because the custom UART interface may be experiencing some
106         # slight clock drifting
107         time.sleep(0.005)
108
109 Lane_Cnt_Dict = {
110     "TL_West": (17, 2),
111     "TL_East": (16, 2)
112 }
113
114 def receive_traffic_state(ser, expected_id):
115     """
116     Reads a length-prefixed packet. Checks ID to prevent crashes.
117     Format: [Len][ID][Statuses...][Pot][ID]
118     """
119     try:
120         # 1. Read Length
121         length_byte = ser.read(1)
122         if not length_byte: return None
123
124         packet_size = int.from_bytes(length_byte, byteorder='big')
125
126         # Sanity Check
127         if packet_size < 5 or packet_size > 64:
128             ser.reset_input_buffer()
129             return None
130
131         # 2. Read Payload
132         data = ser.read(packet_size)
133
134         # Check if we got full data
135         if len(data) != packet_size:
136             return None
137
138         # 3. Validation: Check Start ID against what we expected

```

```

137     start_id_char = data[0:1].decode('ascii')
138
139     if start_id_char != expected_id:
140         return None
141
142     # 4. Extract Data
143     pot_value = int(data[packet_size - 2])
144     status_string = data[1 : packet_size - 2].decode('ascii')
145
146     # 5. Parse Statuses based on ID
147     if start_id_char == 'W':
148         car_lane_cnt = Lane_Cnt_Dict["TL_West"][0]
149         ped_lane_cnt = Lane_Cnt_Dict["TL_West"][1]
150     elif start_id_char == 'E':
151         car_lane_cnt = Lane_Cnt_Dict["TL_East"][0]
152         ped_lane_cnt = Lane_Cnt_Dict["TL_East"][1]
153
154     if len(status_string) < (car_lane_cnt + ped_lane_cnt):
155         return None
156
157     statuses = {
158         "Car_Lights": status_string[0 : car_lane_cnt],
159         "Ped_Lights": status_string[car_lane_cnt : car_lane_cnt + ped_lane_cnt],
160         "Pot_Value": pot_value,
161         "Raw_String": data.hex()
162     }
163     return statuses
164
165 except Exception as e:
166     return None
167
168 def get_traffic_values(Traffic_Values, Lane_Dictionary):
169     """Refreshes the Traffic_Values list with current SUMO data"""
170     Traffic_Values.clear()
171
172     for i in range(len(Lane_Dictionary)):
173         lane_id = Lane_Dictionary.get(i)
174         if lane_id:
175             try:
176                 val_to_send = 0
177                 if i in TL_West_Downstream_Lanes:
178                     occupancy_fraction = traci.lane.getLastStepOccupancy(lane_id)
179                     if(occupancy_fraction >= 90):
180                         val_to_send = occupancy_fraction / 10
181                 else:
182                     val_to_send = traci.lane.getLastStepVehicleNumber(lane_id)
183                     if val_to_send > 254: val_to_send = 254
184                     Traffic_Values.append(int(val_to_send))
185             except traci.exceptions.TraCIException:
186                 Traffic_Values.append(0)
187         else:
188             Traffic_Values.append(0)
189
190 WEST_ROUTES = [
191     ["S_P2_UPSTREAM", "W_POP2_Downstream_E_POP1_Upstream", "N_P1_Upstream"],
192     ["S_P2_UPSTREAM", "W_POP1P2_Downstream", "E1"],
193     ["S_P2_UPSTREAM", "W_POP2_Downstream_E_POP1_Upstream", "E_POP2_Downstream", "E6"],
194     ["S_P2_UPSTREAM", "W_POP1P2_Downstream", "E2"],
195 ]
196
197 VEH_TYPE = "car"
198 vehicle_counter = 0
199 def generate_cars_west(pot_value):
200     global vehicle_counter
201
202     if pot_value <= 0:
203         return
204
205     # Map 0-127 to a probability 0.0 - 1.0
206     spawn_probability = pot_value / 127.0
207
208     if random.random() > spawn_probability:
209         return

```

```

210
211 veh_id = f"west_car_{vehicle_counter}"
212 vehicle_counter += 1
213
214 route_edges = WEST_ROUTES[vehicle_counter % len(WEST_ROUTES)]
215 route_id = f"route_{veh_id}"
216
217 try:
218     traci.route.add(route_id, route_edges)
219     traci.vehicle.add(
220         vehID=veh_id,
221         routeID=route_id,
222         typeID=VEH_TYPE,
223         departLane="best",
224         departSpeed="max"
225     )
226     if DEBUG:
227         print(f"[GEN] Spawned {veh_id} (Pot: {pot_value})")
228
229 except traci.TraCIException as e:
230     print(f"[GEN WEST ERROR] {veh_id}: {e}")
231
232
233 def parse_status_to_sumo_phase(status_dict, sumo_to_hw, ped_lane_cnt):
234     """Converts the Hardware Status Dictionary to a SUMO phase string"""
235     car_string = status_dict["Car_Lights"]
236     peds = status_dict["Ped_Lights"]
237
238     def translate(char):
239         if char == 'G': return 'G'
240         if char == 'Y': return 'y'
241         if char == 'R': return 'r'
242         return 'N'
243
244     phase_chars = []
245
246     for lane_id in sumo_to_hw:
247         status = car_string[lane_id]
248         phase_chars.append(translate(status))
249
250     for i in range(ped_lane_cnt):
251         phase_chars.append(translate(peds[i]))
252     return "".join(phase_chars)
253
254
255 # ==== MAIN SIMULATION LOOP ====
256
257 def main():
258     sumo_cmd = [SUMO_BINARY, "-c", CONFIG_FILE, "--statistic-output", "statistics.xml",
259                 "--duration-log.statistics", "true",
260                 "--summary-output", "summary.xml"]
261 ]
262
263 # Start SUMO
264 try:
265     traci.start(sumo_cmd)
266 except traci.exceptions.FatalTraCIError as e:
267     print(f"FATAL ERROR: {e}")
268     sys.exit(1)
269
270 # Open Serial
271 try:
272     ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1.0)
273     time.sleep(2)
274     print("Serial Port Opened. Starting Simulation...")
275 except serial.SerialException as e:
276     print(f"Error opening serial port: {e}")
277     traci.close()
278     sys.exit(1)
279
280
281 TL_WEST_TRAFFIC_VALUES = []
282 TL_EAST_TRAFFIC_VALUES = []

```

```

283 step = 0
284
285 print(f"Running simulation for {SIM_DURATION} steps...")
286
287 try:
288     while step < SIM_DURATION:
289         start_time = time.time()
290         traci.simulationStep()
291
292         # ===== WEST INTERSECTION =====
293         get_traffic_values(TL_WEST_TRAFFIC_VALUES, TLWest_Lane_Dictionary)
294
295         ser.reset_input_buffer()
296         packet = build_transmission_packet(TL_WEST_TRAFFIC_VALUES)
297         send_data_chunks(ser, packet)
298         status_data_west = receive_traffic_state(ser, 'W')
299
300         if status_data_west:
301             pot = status_data_west["Pot_Value"]
302             if(pot > 0):
303                 generate_cars_west(pot)
304                 new_phase = parse_status_to_sumo_phase(status_data_west,
305                 TL_West_sumo_to_hw_index, 2)
306                 traci.trafficlight.setRedYellowGreenState(TL_ID, new_phase)
307
308         # ===== EAST INTERSECTION =====
309         get_traffic_values(TL_EAST_TRAFFIC_VALUES, TLEast_Lane_Dictionary)
310
311         ser.reset_input_buffer()
312         packet = build_transmission_packet(TL_EAST_TRAFFIC_VALUES)
313         send_data_chunks(ser, packet)
314         status_data_east = receive_traffic_state(ser, 'E')
315
316         if status_data_east:
317             new_phase = parse_status_to_sumo_phase(status_data_east,
318             TL_East_sumo_to_hw_index, 2)
319             traci.trafficlight.setRedYellowGreenState(TL_EAST_ID, new_phase)
320
321         step += 1
322
323         if(FREQUENCY_CONTROL):
324             elapsed = time.time() - start_time
325             if elapsed < 0.3:
326                 time.sleep(0.3 - elapsed)
327
328     except KeyboardInterrupt:
329         print("\nStopping...")
330     finally:
331         traci.close()
332         if ser.is_open:
333             ser.close()
334         print("Simulation Ended.")
335
336 if __name__ == "__main__":
337     main()

```

Listing 1: Scriptul run_simulation.py pentru controlul traficului